

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN**



BÁO CÁO SEMINAR

Kiểm thử cơ sở dữ liệu cho EShop

Giảng viên:

Hồ Tuấn Thanh

Sinh viên:

23127081 - Nguyễn Phan Hùng Linh

23127172 - Nguyễn Chí Đức

23127487 - Thôi Đặng Trường Thọ

Thành phố Hồ Chí Minh, tháng 7 năm 2026

Mục lục

Kiểm thử cơ sở dữ liệu cho EShop

Giới thiệu	3
Tổng quan công cụ	4
Kiểm thử cơ sở dữ liệu quan hệ và NoSQL	5
Cài đặt và thiết lập	7
Thực hành và các bước demo	7
Kịch bản kiểm thử EShop	9
So sánh	11
Tính di động và khả năng tái sử dụng	12
Ưu điểm và hạn chế	14
Các vấn đề đã gặp	15
Bài học rút ra	16
Kết luận	17
Bảng chứng và phụ lục	18
Tài liệu tham khảo	25

1. Giới thiệu

Kiểm thử cơ sở dữ liệu rất quan trọng vì nhiều lỗi ứng dụng không chỉ xuất hiện trên giao diện người dùng. Lỗi có thể xảy ra khi dữ liệu được lưu, cập nhật, xóa hoặc kiểm tra trong cơ sở dữ liệu.

Trong seminar này, hệ thống cần kiểm thử (System Under Test - SUT) là dự án EShop tại <https://github.com/ttbhanh/eshop-sut>

EShop là một SUT phù hợp vì đây là ứng dụng mua sắm thông thường. Hệ thống có người dùng, sản phẩm, danh mục, mã giảm giá, giỏ hàng, đơn hàng và chức năng quản trị. Tất cả chức năng này đều phụ thuộc vào hành vi chính xác của cơ sở dữ liệu.

Ba công cụ được nghiên cứu trong báo cáo gồm:

- DbUnit
- Database Rider
- Tonic.ai

Mục tiêu chính là tìm hiểu cách các công cụ này hỗ trợ chuẩn bị dữ liệu kiểm thử, kiểm tra kết quả trong cơ sở dữ liệu và tạo dữ liệu test an toàn hơn.

2. Tổng quan công cụ

Công cụ	Mục đích chính	Được dùng cho
DbUnit	Thiết lập cơ sở dữ liệu bằng file dataset và kiểm tra dữ liệu	Unit test và integration test cho cơ sở dữ liệu
Database Rider	Giúp viết test theo phong cách DbUnit dễ hơn	Bảo trì database test dễ hơn
Tonic.ai	Tạo dữ liệu test an toàn và che giấu dữ liệu nhạy cảm	Quản lý dữ liệu kiểm thử

DbUnit

DbUnit là thư viện kiểm thử dành cho Java. Công cụ có thể nạp một dataset vào cơ sở dữ liệu trước khi test chạy. Sau khi test thực thi, chúng ta có thể kiểm tra cơ sở dữ liệu có chứa dữ liệu mong đợi hay không.

Trong demo, DbUnit sử dụng dataset XML.

Database Rider

Database Rider được xây dựng trên DbUnit. Công cụ cung cấp cách viết database test gọn hơn. Database Rider có thể sử dụng dataset YAML, thường dễ đọc hơn XML.

Trong demo, Database Rider sử dụng dataset YAML.

Tonic.ai

Tonic.ai khác với DbUnit và Database Rider. Đây không phải là công cụ assertion chính. Tonic.ai được dùng để tạo hoặc che giấu dữ liệu phục vụ kiểm thử.

Đối với EShop, chức năng này hữu ích vì bảng users có thể chứa dữ liệu nhạy cảm như tên, email, mật khẩu, số điện thoại và địa chỉ.

3. Kiểm thử cơ sở dữ liệu quan hệ và NoSQL

Kiểm thử cơ sở dữ liệu có cùng mục tiêu tổng quát trên cả hệ quan hệ và NoSQL: chuẩn bị trạng thái được kiểm soát, thực thi thao tác, xác minh dữ liệu đã lưu và dọn dẹp môi trường test. Tuy nhiên, rủi ro và assertion khác nhau vì hai mô hình tổ chức và phân phối dữ liệu theo những cách khác nhau.

3.1 Kiểm thử cơ sở dữ liệu quan hệ

Kiểm thử cơ sở dữ liệu quan hệ thường kiểm tra:

- Định nghĩa bảng và cột. Ví dụ: bảng users có cột email kiểu TEXT và cột id kiểu số.
- Khóa chính, khóa ngoại, ràng buộc duy nhất, NOT NULL và CHECK. Ví dụ: không cho tạo order với user_id không tồn tại hoặc email bị trùng.
- Hành vi INSERT, UPDATE, DELETE và query. Ví dụ: sau khi checkout, một dòng mới phải được thêm vào bảng orders.
- Phép JOIN và quan hệ giữa các bảng. Ví dụ: kết hợp orders với users để biết order thuộc về ai.
- Giao dịch ACID, COMMIT và ROLLBACK. Ví dụ: nếu trừ tiền kho thất bại thì order cũng phải được hoàn tác.
- Index và SQL execution plan. Ví dụ: query tìm user theo email phải sử dụng index thay vì quét toàn bộ bảng.
- Database migration và khả năng tương thích ngược. Ví dụ: thêm cột mới không được làm hỏng code vẫn đọc schema cũ.
- Stored procedure, trigger và view nếu ứng dụng sử dụng chúng. Ví dụ: trigger tự cập nhật thời gian updated_at sau khi sửa order.

Test trên cơ sở dữ liệu quan hệ thường có thể dựa vào database engine để từ chối dữ liệu không hợp lệ. Ví dụ, foreign key có thể ngăn orders.user_id tham chiếu đến một user không tồn tại.

3.2 Kiểm thử cơ sở dữ liệu NoSQL

NoSQL là một nhóm rộng. Document database, key-value store, wide-column database và graph database có nhu cầu kiểm thử khác nhau. Với document database như MongoDB, hoạt động kiểm thử thường bao gồm:

- Kiểm tra cấu trúc document, field bắt buộc, kiểu dữ liệu, object lồng nhau và mảng. Ví dụ: order phải có status, items phải là một mảng và total phải là số.
- Kiểm tra schema validation nếu cơ sở dữ liệu hỗ trợ. Ví dụ: database từ chối document có status: 123 nếu status chỉ được là chuỗi.
- Kiểm tra document nhúng và reference giữa các collection. Ví dụ: địa chỉ giao hàng được nhúng trong order, còn userId trỏ đến collection users.
- Kiểm tra tính toàn vẹn ở tầng ứng dụng khi không có foreign key. Ví dụ: ứng dụng phải tự từ chối order nếu userId không tồn tại.
- Kiểm tra document query và aggregation pipeline. Ví dụ: lọc order pending, sau đó nhóm theo user và tính tổng tiền.

- Kiểm tra phạm vi cập nhật atomic và hành vi transaction. Ví dụ: tăng số lượng sản phẩm trong giỏ phải được thực hiện trọn vẹn, không cập nhật dở dang.
- Kiểm tra strong consistency hoặc eventual consistency sau khi ghi. Ví dụ: sau khi đổi status thành paid, hệ thống có thể đọc thấy ngay hoặc phải chờ đồng bộ.
- Kiểm tra partition key, shard key, hot partition và phân phối dữ liệu. Ví dụ: dùng userId làm key không tốt có thể khiến một user tạo quá nhiều request trên cùng partition.
- Kiểm tra secondary index, unique index và TTL index. Ví dụ: tìm user theo email phải nhanh, email không được trùng và session hết hạn phải tự xóa.
- Kiểm tra nhiều phiên bản document cùng tồn tại trong quá trình thay đổi schema. Ví dụ: ứng dụng phải đọc được cả document cũ có name và document mới có fullName.
- Kiểm tra retry, idempotency, replication và failover. Ví dụ: gửi cùng request checkout hai lần chỉ tạo một order, kể cả khi server chuyển sang node dự phòng.

Một assertion chạy ngay sau thao tác ghi có thể đúng với strong consistency nhưng không ổn định với eventual consistency. Test nên sử dụng bounded retry và timeout rõ ràng thay vì sleep cố định.

3.3 Khác biệt chính

Vấn đề kiểm thử	Cơ sở dữ liệu quan hệ	Cơ sở dữ liệu NoSQL
Cấu trúc dữ liệu	Bảng, hàng và cột	Collection, document, key hoặc graph
Schema	Thường được khai báo và thực thi rõ ràng	Linh hoạt, tùy chọn hoặc do ứng dụng thực thi
Quan hệ	Foreign key và join	Dữ liệu nhúng hoặc reference do ứng dụng quản lý
Transaction	Giao dịch ACID phổ biến	Phụ thuộc database và phạm vi thao tác
Consistency	Thường là strong consistency trong một database	Strong, eventual hoặc có thể cấu hình
Phân phối	Thường được test như một dịch vụ database	Partition, sharding và replication là rủi ro chính
Thay đổi schema	Migration theo thứ tự	Nhiều phiên bản document có thể cùng tồn tại
Assertion điển hình	Row, constraint, join, transaction	Document shape, consistency, partition, aggregation

3.4 Khả năng áp dụng của ba công cụ

Công cụ	Cơ sở dữ liệu quan hệ	Cơ sở dữ liệu NoSQL
DbUnit	Phù hợp trực tiếp thông qua JDBC và dataset dạng bảng	Không hỗ trợ native cho document hoặc key-value
Database Rider	Phù hợp vì sử dụng DbUnit và JDBC	Không hỗ trợ NoSQL native
Tonic Structural	Hỗ trợ nhiều connector quan hệ	Hỗ trợ một số connector như MongoDB và Amazon DynamoDB

Vì vậy, DbUnit và Database Rider nên được mô tả là công cụ kiểm thử cơ sở dữ liệu quan hệ. Dự án NoSQL thường cần native driver của database, test container hoặc môi trường test riêng, cùng assertion hiểu document, consistency và partition. Tonic Structural có thể chuẩn bị dữ liệu an toàn cho một số hệ NoSQL, nhưng vẫn cần framework kiểm thử riêng để xác minh ứng dụng hoạt

động đúng.

4. Cài đặt và thiết lập

Thiết lập cơ bản được sử dụng cho seminar gồm:

1. Cài Node.js cho backend và frontend của EShop.
2. Cài Java và Maven cho demo DbUnit và Database Rider.
3. Cài công cụ SQLite để kiểm tra cơ sở dữ liệu EShop.
4. Clone hoặc mở dự án EShop.
5. Chạy `npm install` trong các phần Node.js của dự án.
6. Chạy `node database.js` để tạo và seed cơ sở dữ liệu SQLite.
7. Tạo project Maven demo DbUnit.
8. Tạo project Maven demo Database Rider.
9. Chuẩn bị workspace Tonic.ai hoặc quy trình sinh dữ liệu Tonic.ai.

Cơ sở dữ liệu backend của EShop nằm tại:

```
eshop-sut/backend/database.sqlite
```

Backend EShop sử dụng SQLite. File seed chính là:

```
eshop-sut/backend/database.js
```

Cơ sở dữ liệu gồm các bảng như:

- users
- categories
- products
- orders
- order_items
- coupons

5. Thực hành và các bước demo

5.1 Demo DbUnit

Đối với DbUnit, nhóm tạo một project Maven nhỏ.

Các file chính:

```
dbunit-demo/pom.xml
dbunit-demo/src/test/resources/datasets/initial-dataset.xml
dbunit-demo/src/test/java/com/eshop/dbunit/EshopDbUnitTest.java
```

Dataset XML chứa dữ liệu mẫu cho các bảng như users, categories và products.

Test sử dụng DbUnit để:

1. Đọc dataset XML.
2. Khởi tạo lại cơ sở dữ liệu bằng CLEAN_INSERT.
3. Query cơ sở dữ liệu bằng JDBC.
4. Assert dữ liệu mong đợi tồn tại.

Lệnh chạy:

```
mvn test
```

Kết quả ghi nhận:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0  
BUILD SUCCESS
```

Kết quả cho thấy demo DbUnit đã chạy thành công.

5.2 Demo Database Rider

Đối với Database Rider, nhóm tạo một project Maven khác.

Các file chính:

```
database-rider-demo/pom.xml  
database-rider-demo/src/test/resources/datasets/eshop-users.yml  
database-rider-demo/src/test/java/com/eshop/databaserider/EshopDatabaseRiderTest.java
```

Dataset YAML chứa dữ liệu mẫu cho users, categories và products.

Test sử dụng Database Rider để:

1. Đọc dataset YAML.
2. Nạp dataset vào SQLite.
3. Query bảng users.
4. Assert bảng có đúng số lượng row mong đợi.

Lệnh chạy:

```
mvn test
```

Kết quả ghi nhận:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0  
BUILD SUCCESS
```

Kết quả cho thấy demo Database Rider đã chạy thành công.

5.3 Demo Tonic.ai

Quy trình Tonic.ai Structural được thực hiện như sau:

1. Khởi tạo lại cơ sở dữ liệu SQLite của EShop.

2. Export các bảng users, products, coupons và orders sang CSV.
3. Upload các file CSV lên Tonic.ai.
4. Tạo file group.
5. Cấu hình generator cho các cột nhạy cảm.
6. Chạy data generation.
7. Tải các file CSV đã generate.
8. So sánh dữ liệu gốc và dữ liệu đã generate.

Các cột nhạy cảm quan trọng trong bảng users:

Cột	Lý do nhạy cảm
name	Tên người dùng thật
email	Thông tin định danh cá nhân
password	Bí mật đăng nhập
shipping_address	Địa chỉ cá nhân
phone	Số điện thoại cá nhân

Các cột như id, role và foreign key thường nên được giữ ổn định vì ứng dụng cần chúng để bảo toàn quan hệ giữa các bảng.

Trong demo seminar, Tonic.ai được dùng cho dữ liệu users. Kiểm tra chính là so sánh CSV gốc với CSV được generate và xác nhận các giá trị nhạy cảm đã thay đổi.

6. Kịch bản kiểm thử EShop

Demo seminar tập trung vào các kịch bản database testing đơn giản, phù hợp với ba công cụ.

Kịch bản A: Seed và kiểm tra user

Đối với DbUnit và Database Rider, kịch bản là:

Nạp dữ liệu test biết trước vào cơ sở dữ liệu SQLite của EShop và kiểm tra bảng users.

Kịch bản này hữu ích vì đăng nhập, checkout và profile đều cần user hợp lệ. Nếu bảng users sai, nhiều chức năng EShop không thể hoạt động.

Kết quả mong đợi:

1. Cơ sở dữ liệu được đưa về trạng thái biết trước.
2. Dataset tạo hai user: một admin và một user test thông thường.
3. Query xác nhận bảng users có dữ liệu mong đợi.

DbUnit thực hiện bằng dataset XML:

```
src/test/resources/datasets/initial-dataset.xml
```

Database Rider thực hiện cùng ý tưởng bằng dataset YAML:

```
src/test/resources/datasets/eshop-users.yml
```

Kịch bản B: Checkout và kiểm tra đơn hàng

Kịch bản demo ứng dụng EShop:

Đăng nhập bằng user test, thêm sản phẩm vào giỏ hàng, checkout và kiểm tra bảng orders.

Kết quả mong đợi sau checkout:

1. Bảng orders có đơn hàng mới.
2. user_id thuộc về user đang đăng nhập.
3. total_amount khớp với tổng tiền giỏ hàng.
4. status mặc định của đơn hàng là pending.

Có thể kiểm tra cơ sở dữ liệu bằng SQL:

```
SELECT COUNT(*) FROM orders;  
SELECT user_id, total_amount, status FROM orders;
```

Kịch bản C: Che giấu dữ liệu user nhạy cảm

Đối với Tonic.ai, kịch bản là:

Export users từ SQLite sang CSV, upload CSV lên Tonic.ai, che giấu cột nhạy cảm và tải CSV đã generate.

Kết quả mong đợi:

1. name, email, password, shipping_address và phone được thay đổi.
2. Giá trị được generate vẫn giống dữ liệu test thực tế.
3. Các cột quan hệ quan trọng như id được giữ ổn định.
4. Dữ liệu mới có thể dùng an toàn hơn cho demo hoặc staging.

7. So sánh

Tiêu chí	DbUnit	Database Rider	Tonic.ai
Loại công cụ	Thư viện kiểm thử	Wrapper của thư viện kiểm thử	Nền tảng dữ liệu test
Input phổ biến	Dataset XML	Dataset YAML	CSV hoặc nguồn database
Output chính	Kết quả test	Kết quả test	Dữ liệu được generate
Mức độ dễ dùng	Trung bình	Dễ hơn DbUnit	Dễ sau khi làm quen UI
Có thể assert kết quả database	Có	Có	Không, nếu chỉ dùng riêng Tonic
Có thể che giấu dữ liệu nhạy cảm	Không	Không	Có
Cách dùng tốt nhất trong EShop	Reset và kiểm tra trạng thái database	Database test gọn hơn	Dữ liệu test an toàn hơn

DbUnit và Database Rider được dùng để kiểm tra cơ sở dữ liệu có đúng hay không. Tonic.ai được dùng để tạo dữ liệu an toàn hơn. Vì vậy, các công cụ bổ trợ lẫn nhau nhưng không thay thế nhau.

8. Phân tích tính di động và khả năng tái sử dụng

Demo đầu tiên là proof of concept, chưa phải framework kiểm thử có khả năng tái sử dụng cao. Việc review hai class DbUnit và Database Rider cho thấy các phần hard-code sau:

- HSQLDB JDBC URL, username và password được viết trực tiếp trong test class.
- Lệnh CREATE TABLE được nhúng trong mã Java.
- Tên bảng bị cố định là USER_ACCOUNT và PRODUCT.
- Đường dẫn dataset bị cố định trong annotation hoặc test setup.
- SQL query và số row mong đợi là 2 bị cố định.
- Tên package Java và metadata Maven vẫn tham chiếu đến EShop.

8.1 Ước lượng mức tái sử dụng của demo hiện tại

Các giá trị sau là ước lượng kỹ thuật dựa trên cấu trúc code, không phải kết quả benchmark đã đo.

Thành phần	Khả năng tái sử dụng trong project Java quan hệ khác
Maven dependency	80-90%
Annotation DbUnit và Database Rider	80-90%
Mẫu connection và test lifecycle	50-70%
Mẫu seed dataset và assertion	60-80%
Schema, tên bảng, SQL và dataset	Gần 0%
Business assertion đặc thù project	Gần 0%

Tổng thể, khoảng 30-40% code hiện tại có thể copy trực tiếp sang project Java/JDBC khác. Phần tái sử dụng được là cấu hình framework và test lifecycle. Database schema, dataset, query và business assertion phải thay đổi vì chúng mô tả hệ thống mới.

Đối với project NoSQL, gần như không thể tái sử dụng trực tiếp phần triển khai DbUnit hoặc Database Rider vì hai công cụ phụ thuộc JDBC và mô hình bảng. Phần có thể tái sử dụng là quy trình cấp cao: chuẩn bị dữ liệu, thực thi thao tác, xác minh trạng thái đã lưu và dọn dẹp.

8.2 Refactor để tăng khả năng tái sử dụng

Test harness cho cơ sở dữ liệu quan hệ nên tách hạ tầng khỏi test case đặc thù nghiệp vụ:

```
database-testing/  
|-- src/test/java/support/  
|   |-- DatabaseConnectionFactory.java  
|   |-- DatabaseTestSupport.java  
|-- src/test/java/tests/  
|   |-- DbUnitDatasetTest.java  
|   |-- DatabaseRiderDatasetTest.java  
|-- src/test/resources/  
|   |-- database-test.properties  
|   |-- schema.sql  
|   |-- datasets/  
|       |-- seed.yml  
|       |-- expected.yml
```

Thông tin kết nối nên lấy từ configuration hoặc biến môi trường thay vì mã Java:

```
db.driver=${DB_DRIVER}  
db.url=${DB_URL}  
db.user=${DB_USER}  
db.password=${DB_PASSWORD}
```

Khi chuyển sang project quan hệ khác, nhóm chỉ cần thay database configuration, schema.sql, dataset và business assertion. Sau refactor, ước lượng 70-80% test harness có thể tái sử dụng trong một project Java quan hệ khác. Domain assertion vẫn phải đặc thù vì nếu biến chúng thành generic thì test sẽ mất ý nghĩa.

9. Ưu điểm và hạn chế

DbUnit

Ưu điểm:

- Kiểm soát mạnh trạng thái cơ sở dữ liệu.
- Phù hợp để học nền tảng database testing.
- Có thể reset dữ liệu trước mỗi test.

Hạn chế:

- Dataset XML có thể dài.
- Mã thiết lập ở mức thấp hơn.
- Người dùng cần hiểu Java, JDBC và bảng trong database.
- Không hỗ trợ native cho NoSQL document hoặc key-value.

Database Rider

Ưu điểm:

- Dataset YAML dễ đọc hơn.
- Giảm một phần mã thiết lập so với DbUnit thuần.
- Phù hợp cho database test cần bảo trì lâu dài.

Hạn chế:

- Vẫn yêu cầu Java và Maven.
- Vẫn cần hiểu khái niệm DbUnit.
- Không thay thế việc hiểu database schema.
- Không hỗ trợ NoSQL native vì vẫn dùng DbUnit và JDBC.

Tonic.ai

Ưu điểm:

- Có thể che giấu dữ liệu nhạy cảm.
- Có thể tạo dữ liệu test trông thực tế.
- Hữu ích khi không nên dùng trực tiếp dữ liệu production.
- Hỗ trợ cơ sở dữ liệu quan hệ và một số connector NoSQL.

Hạn chế:

- Không phải công cụ assertion.
- Cần cấu hình generator chính xác.
- Nếu ID hoặc foreign key thay đổi sai, quan hệ có thể bị phá vỡ.
- Nếu email hoặc password bị che giấu, tài khoản demo mặc định có thể không đăng nhập được.
- Mức hỗ trợ và tính năng NoSQL phụ thuộc connector và license.

10. Các vấn đề đã gặp

Vấn đề	Giải thích ngắn
Thiếu module sqlite3	Dependency Node.js chưa được cài đặt
Lỗi parse Maven pom.xml	Nội dung không hợp lệ bị copy thêm vào cuối pom.xml
DbUnit order test thất bại	Order mong đợi chưa tồn tại
Tonic.ai báo nhiều schema	Các CSV có schema khác nhau bị upload vào cùng một file group
orders.csv rỗng	Chưa có order sau khi reset seed data
Email/password bị mask ảnh hưởng đăng nhập	Credential được generate không còn khớp với luồng demo
Lỗi import/package Database Rider	Tên import Java phải khớp với phiên bản Database Rider

11. Bài học rút ra

Database testing cần dữ liệu ổn định. Nếu cơ sở dữ liệu thay đổi ngẫu nhiên, test sẽ khó đáng tin cậy.

DbUnit và Database Rider hữu ích khi cần chuẩn bị trạng thái database biết trước trước khi test. Hai công cụ cũng hữu ích khi cần kiểm tra database sau một hành động.

Tonic.ai hữu ích khi cần dữ liệu test nhưng không nên dùng dữ liệu nhạy cảm thật. Công cụ bảo vệ quyền riêng tư bằng cách thay thế tên, email, địa chỉ và số điện thoại.

Quan hệ foreign key rất quan trọng. Ví dụ, nếu `orders.user_id` tham chiếu đến `users.id`, cần cẩn thận khi generate hoặc thay đổi ID.

Database test có thể xác minh điều mà screenshot UI không chứng minh được. Ví dụ, sau checkout, UI có thể báo thành công nhưng database test mới xác nhận row thật sự đã được thêm vào bảng `orders`.

Relational và NoSQL có cùng test lifecycle nhưng failure model khác nhau. NoSQL đòi hỏi kiểm tra thêm consistency, partition, replication, document version và quy tắc toàn vẹn ở tầng ứng dụng.

12. Kết luận

DbUnit hữu ích để hiểu nền tảng database testing. Công cụ cho thấy cách nạp dataset và kiểm tra kết quả trong cơ sở dữ liệu.

Database Rider dễ bảo trì hơn vì dataset YAML gọn và workflow test ngắn hơn.

Tonic.ai hữu ích để tạo dữ liệu test an toàn. Công cụ giúp generate hoặc mask dữ liệu để quá trình kiểm thử không làm lộ thông tin nhạy cảm.

Đối với EShop, ba công cụ phối hợp tốt theo các vai trò khác nhau:

- DbUnit và Database Rider giúp xác minh hành vi cơ sở dữ liệu.
- Tonic.ai giúp chuẩn bị dữ liệu test an toàn hơn.
- Kịch bản checkout EShop cho thấy lý do cần kiểm tra database sau hành động của user.

Seminar cũng làm rõ một ranh giới quan trọng: DbUnit và Database Rider là công cụ cho cơ sở dữ liệu quan hệ, trong khi Tonic Structural hỗ trợ cơ sở dữ liệu quan hệ và một số connector NoSQL. Giải pháp có tính di động nên tái sử dụng test harness và workflow, đồng thời giữ schema, dataset và business assertion riêng cho từng project.

13. Bằng chứng và phụ lục

Tài liệu hỗ trợ:

- User Guide
- Hướng dẫn DbUnit từng bước
- Hướng dẫn Database Rider từng bước
- Hướng dẫn kiểm thử Tonic.ai
- AI Audit Report

Bằng chứng screenshot seminar:

Bằng chứng DbUnit

Bằng chứng DbUnit 1

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.371 s -- in com.eshop.dbunit.EshopDbUnitTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.953 s
[INFO] Finished at: 2026-07-03T22:44:51+07:00
[INFO] -----
jduckiess@192 dbunit-demo %
```

Bảng chứng DbUnit 2

EShop

Giỏ hàngChào, Test UserThoát

Thanh toán thành công!

Cảm ơn bạn đã mua sắm tại EShop.

[Quay lại trang chủ](#)

© 2026 EShop SUT. Dành cho mục đích kiểm thử.

Bảng chứng DbUnit 3

```
dbunit-demo > src > test > java > com > eshop > dbunit > J EshopDbUnitTest.java
18  class EshopDbUnitTest {
24      void shouldLoadInitialDatasetAndCheckUsers() throws Exception {
37
38          try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + DB_PATH);
39              Statement statement = connection.createStatement();
40              ResultSet resultSet = statement.executeQuery("SELECT COUNT(*) AS total FROM users")) {
41
42              assertEquals(2, resultSet.getInt("total"));
43          }
44      }
45      @Test
46      void shouldCheckOrdersAfterCheckout() throws Exception {
47          try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + DB_PATH);
48              Statement statement = connection.createStatement();
49              ResultSet resultSet = statement.executeQuery("SELECT COUNT(*) AS total FROM orders")) {
50
51              int totalOrders = resultSet.getInt("total");
52
53              System.out.println("Total orders: " + totalOrders);
54
55              // Sau khi checkout thành công, totalOrders nên lớn hơn 0
56              assertEquals(1, totalOrders);
57          }
58      }
59  }
```

Bảng chứng DbUnit 4

```
jiduckiess@192 SeminarTesting % cd /Users/jiduckiess/Documents/SeminarTesting/dbunit-demo
mvn test
jiduckiess/.m2/repository/org/xerial/sqlite-jdbc/3.45.3.0/sqlite-jdbc-3.45.3.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

Total orders: 1
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.387 s -- in com.eshop.dbunit.EshopDbUnit
Test
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.224 s
[INFO] Finished at: 2026-07-04T00:17:01+07:00
```

Bảng chứng Database Rider

Bảng chứng Database Rider 1

```
jiduckiess@192 backend % cd /Users/jiduckiess/Documents/SeminarTesting/data
base-rider-demo
mvn test
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for call
ers in this module
WARNING: Restricted methods will be blocked in a future release unless nati
ve access is enabled

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.60
6 s -- in com.eshop.databaserider.EshopDatabaseRiderTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.251 s
[INFO] Finished at: 2026-07-04T18:24:40+07:00
[INFO] -----

jiduckiess@192 database-rider-demo %
```

Bảng chứng Database Rider 2

```
jiduckiess@192 database-rider-demo % cd /Users/jiduckiess/Documents/Seminar
Testing
sqlite3 backend/database.sqlite "SELECT id, name, email, role FROM users;"
1|Admin User|admin@eshop.com|admin
2|Test User|test@eshop.com|user
jiduckiess@192 SeminarTesting %
```

Ln 280, Col 75

Bảng chứng Tonic.ai

Bằng chứng Tonic.ai 1

```
jiduckiess@192 SeminarTesting % sqlite3 -header -csv backend/database.sqlite "SELECT * FROM users;" > tonic-export/source/users.csv
sqlite3 -header -csv backend/database.sqlite "SELECT * FROM products;" > tonic-export/source/products.csv
sqlite3 -header -csv backend/database.sqlite "SELECT * FROM coupons;" > tonic-export/source/coupons.csv
sqlite3 -header -csv backend/database.sqlite "SELECT * FROM orders;" > tonic-export/source/orders.csv
jiduckiess@192 SeminarTesting % ls -la tonic-export/source
total 24
drwxr-xr-x@ 6 jiduckiess  staff  192 Jul  4 12:49 .
drwxr-xr-x@ 3 jiduckiess  staff   96 Jul  4 12:49 ..
-rw-r--r--@ 1 jiduckiess  staff  256 Jul  4 12:49 coupons.csv
-rw-r--r--@ 1 jiduckiess  staff    0 Jul  4 12:49 orders.csv
-rw-r--r--@ 1 jiduckiess  staff  700 Jul  4 12:49 products.csv
-rw-r--r--@ 1 jiduckiess  staff  194 Jul  4 12:49 users.csv
id,name,email,password,role,login_attempts,locked_until,reset_token,shipping_address,phone
1,"Admin User",admin@eshop.com,Admin123!,admin,0,,,,,
2,"Test User",test@eshop.com,Test1234!,user,0,,,,,
```

Bằng chứng Tonic.ai 2

```
tonic-export > source > users.csv
1 id,name,email,password,role,login_attempts,locked_until,reset_token,shipping_address,phone
2 1,"Admin User",admin@eshop.com,Admin123!,admin,0,,,,,
3 2,"Test User",test@eshop.com,Test1234!,user,0,,,,,
4
```

Bằng chứng Tonic.ai 3

The screenshot shows the Tonic.ai interface. At the top, there's a navigation bar with options like 'Search workspaces', 'Workspaces', 'Add Data Source', 'Getting Started', 'Book A Demo', and '13 days left in free trial'. Below this, the workspace name 'Duc' is displayed, along with 'Owned by Đức Nguyễn Chí'. A sidebar on the left contains a 'Hello!' message and a list of actions: 'Review my sensitivity scan results', 'Configure generators to mask sensitive columns', and 'What can Tonic Structural do?'. The main area shows a message 'Could not connect to source database' with a sub-message 'To connect a data source, edit the workspace.' and two buttons: 'Edit Workspace Settings' and 'View Docs'. A red-bordered box contains the error message: 'Connection Error No Source DB has been configured for this Workspace'.

Bảng chứng Tonic.ai 4

Model		Table: users	De-Identify		Preview		
NAME		id	name	email	password	role	login_attempts
name » Name Type : First Last		text	text	text	text	text	text
EMAIL							
email » Email							
PASSWORD							
password » Constant Value : MaskedPassword123!							
SHIPPING_ADDRESS							
shipping_address » Address Type : Zip Code							
PHONE							
phone » Phone							

Bảng chứng Tonic.ai 5

Confirm Generation

Settings

Destination

Generation removes previous versions of the generated files, and creates new versions of the files based on the current generator settings.

☐ Enable Diagnostic Logging

☐ Send Logs to Tonic.ai

☐ Collect Performance Metrics

Want faster generations?

Use the suggested tips to make workspace and database changes that can improve generation performance.

[Generation Tips](#)

Bảng chứng Tonic.ai 6

Agent

File Groups

Privacy Hub

Database View

Table View

Foreign Keys

Jobs

Schema Changes

Post-Job Actions

Workspace Settings

Jobs

The jobs are grouped by category. To view the jobs in a category, click the category tab.

Learn More

Scheduled Jobs

Schedule your data generation jobs to run at the times that work best for you.

Create Schedule

All Jobs

Data Generation

Sensitivity Scan

Schema

Filter jobs by ID...

Job ID	Type	Status	Submitted	Completed
088f3138-35fc-a97e-abee... Data Pipeline Generation	Schema Retrieval	Completed a few seconds ago	07/04/2026 5:45 PM	07/04/2026 5:45 PM
e611fb86-32e3-9a33-8a3...	Data Pipeline Generation	Canceled a minute ago	07/04/2026 5:44 PM	07/04/2026 5:45 PM
962c9422-6f22-36b6-003...	Data Pipeline Generation	Completed a few seconds ago	07/04/2026 5:43 PM	07/04/2026 5:45 PM
e066ee58-1b35-846d-00c...	Schema Retrieval	Completed 18 minutes ago	07/04/2026 5:28 PM	07/04/2026 5:28 PM
71ca4e98-8cec-6bde-158...	Sensitivity Scan	Completed 16 minutes ago	07/04/2026 5:28 PM	07/04/2026 5:29 PM
1bf1d0a4-ad3d-775d-9d94...	Schema Retrieval	Completed 18 minutes ago	07/04/2026 5:27 PM	07/04/2026 5:27 PM
5a8339f9-ac64-c764-167...	Sensitivity Scan	Completed 17 minutes ago	07/04/2026 5:27 PM	07/04/2026 5:29 PM
2f97e335-0de5-cd11-525e...	Schema Retrieval	Completed 24 minutes ago	07/04/2026 5:22 PM	07/04/2026 5:22 PM
5162093f-c0a8-b751-4a66...	Sensitivity Scan	Completed 23 minutes ago	07/04/2026 5:22 PM	07/04/2026 5:22 PM
514a4f8e-10d2-a664-e1d7...	Schema Retrieval	Completed 25 minutes ago	07/04/2026 5:20 PM	07/04/2026 5:20 PM

View 10 jobs per page

Showing 1 to 10 of 12

Bảng chứng Tonic.ai 7

Back to All Jobs

Data Pipeline Generation Job Details

Job ID: 962c9422-6f22-36b6-003d-5e8644eb64f3

Copy Job ID

Reports and Logs

Workspace

Untitled Workspace

Owner

ncduc23@clc.fitus.edu.vn

DB Type

Files

Subsetting

Not In Use

Post-Job Scripts

Not In Use

Webhooks

Not In Use

Worker Version

1795

Data available for file groups

De-identified files from this data generation job are available for download. Select a file group to download the files.

Download Results

Filter...

coupons

products

orders

users

Start Time

5:45:29 PM

Duration

0m 9s

Job Completed

Completed in a few seconds

Executing test suites

Completed in a few seconds

Showing slowest 2 tasks

Executing Source test suites completed in < 1 second

Executing Destination test suites completed in < 1 second

Total Time: < 1 second

Running pre-job checks

Completed in a few seconds

Applying generators to chained Fks

Completed in a few seconds

Copying passthrough table

Completed in a few seconds

Showing slowest 2 tasks

Copying File Group "coupons" completed in < 1 second

Bằng chứng Tonic.ai 8

```
2, test user, test@eshop.com, test123!, user, 0, , , ,
● jiduckiess@192 SeminarTesting % ls -la tonic-export/generated
head -n 5 tonic-export/generated/users.csv
total 32
drwxr-xr-x@ 6 jiduckiess  staff  192 Jul  4 17:50 .
drwxr-xr-x@ 4 jiduckiess  staff  128 Jul  4 17:50 ..
-rw-r--r--@ 1 jiduckiess  staff  256 Jul  4 17:50 coupons.csv
-rw-r--r--@ 1 jiduckiess  staff  106 Jul  4 17:50 orders.csv
-rw-r--r--@ 1 jiduckiess  staff  688 Jul  4 17:50 products.csv
-rw-r--r--@ 1 jiduckiess  staff  228 Jul  4 17:50 users.csv
id,name,email,password,role,login_attempts,locked_until,reset_token,shipping_address,phone
1,Rebecca  Cobler,mjwp@objcs.ndn,MaskedPassword123!,admin,0,,,97708,
2,Leigha  Batcher,xytg@zxfgo.goy,MaskedPassword123!,user,0,,,99547,
○ jiduckiess@192 SeminarTesting %
```

Bằng chứng Tonic.ai 9

```
tonic-export > generated > users.csv
1  id,name,email,password,role,login_attempts,locked_until,reset_token,shipping_address,phone
2  1,Rebecca  Cobler,mjwp@objcs.ndn,MaskedPassword123!,admin,0,,,97708,
3  2,Leigha  Batcher,xytg@zxfgo.goy,MaskedPassword123!,user,0,,,99547, Pin selection to
```

Các lệnh được sử dụng để tạo bằng chứng:

```
cd dbunit-demo
mvn test
```

```
cd database-rider-demo
mvn test
```

```
cd eshop-sut/backend
node database.js
```

14. Tài liệu tham khảo

- DbUnit: <https://www.dbunit.org/>
- Database Rider: <https://database-rider.github.io/database-rider/>
- Tonic.ai: <https://www.tonic.ai/>
- Tonic Structural data connectors:
<https://docs.tonic.ai/app/setting-up-your-database/database-connectors>
- Tonic Structural MongoDB support:
<https://docs.tonic.ai/app/setting-up-your-database/mongodb>